RA23E01 – ROBOT OPERATING SYSTEM

AGILEX SCOUT MINI NAVIGATION USING ROS 2 WITH WHEEL-SLIP SIMULATION AND CORRECTION (SECOND REVIEW)

Submitted By:

SAILAKSHMI BT (2024511019)

BACHELOR OF ENGINEERING

in

ROBOTICS AND AUTOMATION



DEPARTMENT OF PRODUCTION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY
ANNA UNIVERSITY - CHENNAI 600044

BONAFIDE CERTIFICATE

Certified that this project report “AgileX Scout Mini Navigation using ROS 2 with Wheel-Slip Simulation and Correction (Second Review)” is the Bonafide work of Sailakshmi BT (2024511019) who carried out the project work under my supervision.

SIGNATURE

SUPERVISOR

TABLE OF CONTENTS:

S.No	TOPICS	PAGE
1	**Introduction**	5
2	**Work Done Since the First Review**	5
3	**Robot Data and Assumed Parameters**	6
4	**DC Motor, Gearbox and Battery Model**	8
5	**Torque and Force for Driving**	11
6	**Traction Limit and Wheel Slip**	12
7	**Slip Detection and Correction**	13
8	**The Slip World in Gazebo**	15
9	**Ground Station and Direct Routing**	16
10	**Software Changes**	17
11	**Test Method**	20
12	**Results**	20
13	**Unit Tests**	24
14	**Problems Solved**	24
15	**Algorithms Used**	26
16	**How to Run**	27
17	**Limitations and Future Enhancements**	28
18	**Conclusion**	28
19	**References**	28

Abstract

This report presents the second review of the AgileX Scout Mini simulation in ROS 2 Jazzy and Gazebo Harmonic. The first review showed the robot navigating a civil facility on a floor with good grip. The second review adds what happens when the floor does not give grip: a second world with three flush low-friction patches ($\mu = 0.04$), a slip_control node that estimates the true ground speed from the IMU, detects slip, limits the wheel command, holds the heading and publishes corrected odometry, and a new drive-motor layer (motor_model) that calculates the torque, current, power and battery state of the four 150 W hub motors of the Scout Mini.

The motor layer is a calculation layer: the Gazebo DiffDrive plugin still follows the velocity command, so the motor model computes, for every command, the wheel force and torque that the real robot would need, the motor current and the battery voltage, and compares the force demand with the grip limit $\mu\ m\ g$ of the floor. For the simulated mass of 28.3 kg the cruise torque is 1.02 N·m per wheel (20 % of the rated 5.0 N·m), a 1 m/s² start needs 1.58 N·m and the grip limit on a patch is only 0.22 N·m. The hand calculation and the live simulation agree to within 0.01 N·m at cruise (0.89 N·m at 0.5 m/s) and the start peak of 3.16 N·m calculated lies in the measured range of 2.7–3.2 N·m. Six unit tests of the model pass.

Twenty-leg patrol runs on the normal floor completed in all four runs (mean ground-truth error 0.23 m). On the slip floor with the correction on, 3 of 4 runs completed; with the correction off, 2 of 3 completed and the robot overshot the corridor hub in every run. All results are from simulation.

How the slip is corrected, in one line: the IMU gives the true ground speed, and when the wheels turn faster than that the controller limits the wheel command to the true speed and corrects the odometry before the EKF uses it.

1. Introduction

The first review presented the Scout Mini navigating a 20 m × 16 m civil facility with six rooms in Gazebo. A SolidWorks model was converted to URDF (about 28 kg, four driven wheels), the facility and its occupancy map were generated, and Nav2 (EKF, AMCL, Smac 2D planner, regulated pure pursuit) drove the robot through the rooms from a PyQt5 Ground Station. In that work the floor always gave good grip, the Gazebo DiffDrive plugin turned the velocity command into wheel speed exactly, and the robot could neither lose traction nor run out of motor torque.

A real robot meets wet or polished patches, and its motors can only deliver a limited torque. When a wheel slips, the wheel odometry counts distance that was never travelled and localisation drifts. The second review therefore asks two questions: what does the drive have to deliver, and what happens when the floor cannot take it? The objectives are:

- to add the real drive of the Scout Mini (four 150 W brushless hub motors, a 24 V battery) as a calculation layer with a torque and current limit, and to calculate the wheel force, torque, current and power for driving cases;
- to calculate the traction limit μ m g of the floor and to compare it with the force demand, which shows when slip must occur;
- to add a second world with low-friction patches and to show wheel slip;
- to estimate the true speed, detect slip and correct it (traction limiting, heading hold, odometry correction) in a slip_control node, with a switch to turn the correction off for comparison;
- to show slip in the Ground Station, to send each click as one direct shortest-path goal, and to measure everything again with each number traced to a log.

Section 2 lists what changed. Sections 3 to 6 give the assumptions, the motor, gearbox and battery model and the calculations of torque, force and traction. Sections 7 to 10 describe the slip controller, the slip world, the Ground Station and the software. Sections 11 to 13 give the test method, the results and the unit tests; Sections 14 to 18 the problems solved, the algorithms, how to run, limitations and conclusion.

2. Work Done Since the First Review

The table compares the two reviews. The robot model, the facility, localisation and the Nav2 stack of the first review are unchanged in kind; the floor, the drive-motor layer, the slip control, the click routing and the measurements are new.

Table 1: Changes between the first and the second review.

Item	First review	Second review (this report)
Floor	Uniform floor, μ = 0.25; wheels always grip	Second world with three flush patches, μ = 0.04
Wheel drive	DiffDrive plugin: wheel speed set directly; no motor	Same plugin, fed through motor_model: torque, current, power, battery and grip limit calculated for every command
Motor data	None	4 × 150 W brushless hub motors (manual), K_t = 0.64 N·m/A, 24 V, 1 : 1 gearbox (assumed), peak torque 15 N·m
Battery	None	24 V, 15 Ah, 6S: open-circuit voltage by charge, internal resistance 0.08 Ω
Slip	Not possible	Estimated from the IMU; traction limiter, heading hold, odometry correction; switch for comparison

Item	First review	Second review (this report)
Click routing	Each click split into door-aligned waypoints	One direct NavigateToPose goal; route_id pre-emption; goal snapped ≥ 0.55 m from obstacles
Nav2 costs / tolerance	cost_travel_multiplier 2.0; yaw tolerance 0.35 rad	Multiplier 1.0 (shortest path); yaw tolerance 3.14 rad (position only)
Ground Station	Camera, map, click-to-navigate	Adds slip panel and correction-mode label
Tests	One patrol run with ground-truth check	20-leg patrols on both floors, route tours, six unit tests of the motor model, headless launch for tests

3. Robot Data and Assumed Parameters

The Scout Mini is a four-wheel skid-steer robot. Its manual (v1.0.1) gives four 150 W DC brushless hub motors; it does not publish the motor constants, gear ratio or battery resistance. The simulated robot is the SolidWorks-derived URDF, which is heavier than the real 20 kg robot. Quantities that were taken from a document or the URDF are marked as such; every other value is an assumption, chosen to be plausible for a 24 V drive and listed with its reason.

1.2 Performance parameters

Parameter Types	Items	Values
Mechanical specifications	L × W × H (mm)	627 × 550 × 252
	Wheelbase (mm)	452
	Front/rear wheel base (mm)	450
	Weight of vehicle body (kg)	20
	Battery type	Lithium battery 24V 15aH
	Motor	DC brushless 4 X 150W
	Drive type	Independent four-wheel drive
	Suspension	Independent suspension with rocker arm
	Steering	Four-wheel differential steering
	Safety equipment	Servo brake/anti-collision tube
Motion	No-load highest speed (km/h)	≤20
	Minimum turning radius	Be able to turn on a pivot
	Maximum climbing capacity	≥30°
	Minimum ground clearance (mm)	107

Fig. 1: Specification of the AgileX Scout Mini (manual v1.0.1): four 150 W DC brushless motors, 24 V battery, 20 kg.

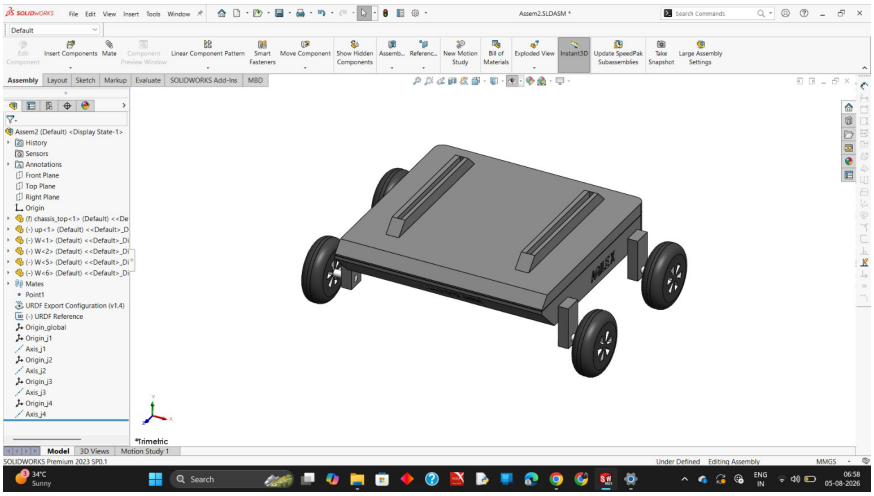


Fig. 2: SolidWorks assembly of the Scout Mini that was converted to the URDF used in Gazebo.

Table 2: Robot data of the simulated Scout Mini.

Robot quantity	**Value**	**Source / note**
Mass m (simulated)	28.3 kg	Sum of the URDF link masses; real robot 20 kg
Wheel radius r	0.08 m	URDF
Wheel separation B	0.612 m	DiffDrive plugin (skid-steer treated as differential drive)
Wheel base L	0.463 m	URDF
Yaw inertia I_z	1.6 kg·m²	Estimated from the URDF boxes
Weight m g	277.6 N	m g with g = 9.81 m/s²
Rolling resistance c_{rr}	0.03	Assumed, hard rubber on tile
Turning friction μ_t	0.35	Assumed, skid-steer scrub
Joint friction / damping	0.1 N·m / 0.1 N·m·s/rad	URDF
Drag area C_dA	0.1925 m²	Assumed (0.35 × 0.55); negligible at 0.6 m/s (0.04 N)
Wheel–floor friction, dry / patch	0.25 / 0.04	URDF contact tags / slip world
Plugin limits	±2.5 m/s, ±3.5 rad/s; ±4.0 m/s², ±6.0 rad/s²	DiffDrive plugin
Nav2 limits	0.6 m/s cruise (max 0.7 m/s); ω ≤ 1.0 rad/s; accel 1.0, decel −1.5 m/s²; angular accel 6.0 rad/s²	nav2_params.yaml

Table 3: Motor, gearbox and battery parameters used in Section 4.

Motor / battery parameter	**Symbol**	**Value**	**Reason (assumed unless stated)**
Motors	–	4 × 150 W, DC brushless hub	Manual (official)
Battery voltage	V	24 V (25.2 V full, 21.0 V empty)	Manual; 6S pack
Battery capacity	–	15 Ah	Datasheet value of the Scout Mini
Battery internal resistance	R_{int}	0.08 Ω	Assumed for a 6S pack
Torque / back-EMF constant	$K_t = K_e$	0.64 N·m/A	24 V / 37.5 rad/s, so that the no-load speed is 3.0 m/s
Winding resistance	R	0.6 Ω	Assumed
Rated / peak current	I_{rated}, I_{peak}	7.8 A / 23.4 A (3×)	150 W at 30 rad/s gives 5.0 N·m, so I_{rated} = 5.0 / 0.64; peak limit of the driver
Rated / peak torque	τ_{rated}, τ_{peak}	5.0 / 15.0 N·m	K_t I
Gear ratio / efficiency	N, η	1 : 1, 1.0	Direct-drive hub motor; manual lists no gearbox
Idle electronics power	P_0	12 W	Assumed

Actuators and sensors of the simulated robot

Each of the four wheels is a continuous joint (j1 and j2 on the left, j3 and j4 on the right). The DiffDrive plugin turns the velocity command (v, ω) into wheel joint velocities, so for straight driving the wheel speed is v / r: 7.5 rad/s at the Nav2 cruise speed of 0.6 m/s. In a turn at v = 0.6 m/s and ω = 1.0 rad/s the outer wheels turn at (v + ωB/2) / r = 11.3 rad/s and the inner wheels at 3.7 rad/s. All sensors are Gazebo sensors defined in the URDF and bridged to ROS 2 by ros_gz_bridge; the noise values are those written in the URDF.

Table 4: Sensors of the simulated robot.

Sensor	Gazebo type, topic	Rate	Range / resolution	Noise	Used by
2D LiDAR	gpu_lidar, /scan	20 Hz	720 samples over −π..π; 0.12–16 m	σ = 0.01 m	AMCL, costmaps, SLAM Toolbox
IMU	imu, /imu	100 Hz	3-axis gyroscope and accelerometer	gyro σ = 0.0002 rad/s; accel σ = 0.017 m/s²	EKF (yaw rate); slip_control
RGB-D camera	rgbd_camera, /camera/image_raw	30 Hz	640 × 480; field of view 80°; 0.1–10 m	none set	Ground Station view only
Wheel odometry	DiffDrive, /odom	50 Hz	integrates the commanded wheel motion	none (ideal wheels; slip changes the true motion, not the reading)	EKF; slip_control

The IMU gives the yaw rate (wheel-derived yaw drifts on a skid-steer base; about 60–65 % of the commanded yaw was measured before the IMU was fused) and the accelerations used by the slip estimator. Wheel odometry gives the short-term motion estimate; on a slippery floor it over-reports, which is the error slip_control corrects. The EKF runs at 20 Hz in 2D mode; AMCL uses 1000–4000 particles with a likelihood-field model.

4. DC Motor, Gearbox and Battery Model

The model is built from the assumptions below. Each assumption is numbered so that the calculation can be checked against it.

Assumptions of the motor model	
M1	The four wheels share the load equally; the wheels on one side carry the same torque.
M2	Each wheel is driven by one DC motor whose torque is K_t I and whose back-EMF is K_e ω_m, with $K_t = K_e$ in SI units.
M3	The gearbox has ratio N and efficiency η; for the hub motor N = 1 and η = 1.
M4	The motor driver limits the current to I_{peak} and the battery voltage limits the speed (no-load speed V / K_e).
M5	The battery has an open-circuit voltage that falls with charge and an internal resistance R_{int}.
M6	The friction of the wheel joints is a constant plus a viscous term (0.1 N·m and 0.1 N·m·s/rad).
M7	The Gazebo DiffDrive plugin enforces the velocity command; the model only reduces acceleration when the torque limit is reached, and otherwise reports what the command would need.

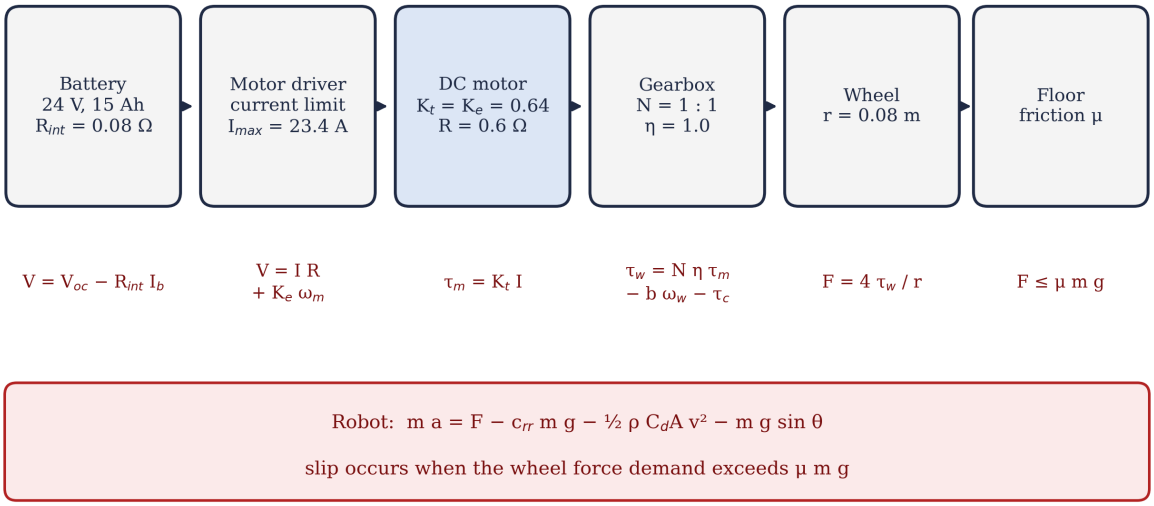


Fig. 3: Drive train of one wheel and the equations of each block.

A. Electrical and mechanical equations

The wheel angular speed follows from the body speed v and the yaw rate ω, with B the wheel separation:

$$\omega_{w,L} = \frac{v - \omega B/2}{r}, \qquad \omega_{w,R} = \frac{v + \omega B/2}{r} \tag{1}$$

The torque on one wheel is the part that accelerates and drives the body plus the joint losses:

$$F = m\,a + c_{rr}\,m\,g\cos\theta + m\,g\sin\theta + \frac{1}{2}\rho\,C_d A\,v|v| \tag{2}$$

$$M = I_z\,\alpha + \mu_t\,m\,g\,\frac{L}{2}\,\mathrm{sgn}(\omega) \tag{3}$$

$$F_{L,R} = \frac{F}{2} \mp \frac{M}{B} \tag{4}$$

$$\tau_w = \frac{F_{side}\,r}{2} + \tau_f + b\,\omega_w \tag{5}$$

Equation (2) is the force along the drive; the slope term is zero on the flat floor of this project (θ = 0) and is kept only for the check against the manual. Equation (3) is the yaw moment needed to turn. In (5) F_{side} is the force of one side, shared by its two wheels. The motor current and the terminal voltage follow from M2 and M3:

$$I = \frac{\tau_w}{N\eta K_t} \tag{6}$$

$$V = K_e N \omega_w + R I \tag{7}$$

$$\tau_{max} = N\eta K_t \min\left(I_{peak},\ \frac{V_{bat} - K_e N|\omega_w|}{R}\right) \tag{8}$$

Equation (8) gives the torque that is available at a wheel speed ω_w: the current limit holds the torque at 15.0 N·m up to the knee at 15.6 rad/s (1.25 m/s) and the voltage then lowers it to zero at 37.5 rad/s (3.0 m/s), as in Fig. 4. The battery obeys:

$$V_{bat} = V_{oc}(SOC) - R_{int}\,I_{bat}, \qquad I_{bat} = \frac{P}{V_{bat}} \tag{9}$$

The driver limits the current to $I_{max} = I_{peak}$. The largest wheel torque, the force of the four wheels at the ground, the no-load speed, the speed at which the current limit ends and the unlimited stall torque are:

$$\tau_{w,max} = N \eta K_t I_{max}, \qquad F_{max} = \frac{4\,\tau_{w,max}}{r} \tag{10}$$

$$\omega_0 = \frac{V}{K_e N}, \qquad v_0 = r\,\omega_0, \qquad \omega_c = \frac{V - I_{max} R}{K_e N} \tag{11}$$

$$\tau_{stall} = \frac{N \eta K_t V}{R} \tag{12}$$

Calculation (N = 1, η = 1, $K_t = K_e$ = 0.64 N·m/A, R = 0.6 Ω, V = 24 V, I_{max} = 23.4 A, r = 0.08 m):

$$\tau_{w,max} = 1 \cdot 1 \cdot 0.64 \cdot 23.4 = 15.0 \text{ N} \cdot \text{m}, \qquad F_{max} = \frac{4 \cdot 14.98}{0.08} = 749 \text{ N} \tag{13}$$

$$\omega_0 = \frac{24}{0.64 \cdot 1} = 37.5 \text{ rad/s}, \qquad v_0 = 0.08 \cdot 37.5 = 3.00 \text{ m/s} \tag{14}$$

$$\omega_c = \frac{24 - 23.4 \cdot 0.6}{0.64} = 15.6 \text{ rad/s}, \qquad v_c = 0.08 \cdot 15.6 = 1.25 \text{ m/s} \tag{15}$$

$$\tau_{stall} = \frac{1 \cdot 1 \cdot 0.64 \cdot 24}{0.6} = 25.6 \text{ N} \cdot \text{m} \;\; \text{(limited to 15.0)} \tag{16}$$

The current limit therefore decides the largest torque: 15.0 N·m per wheel and 749 N for the four wheels, up to 1.25 m/s. Above that speed the back-EMF takes over and the torque falls along the line of Eq. (8) (Fig. 4), reaching zero at 3.0 m/s. The torque needed for each driving case is also drawn in Fig. 4: where a point lies above the curve, the motor cannot supply it (none does).

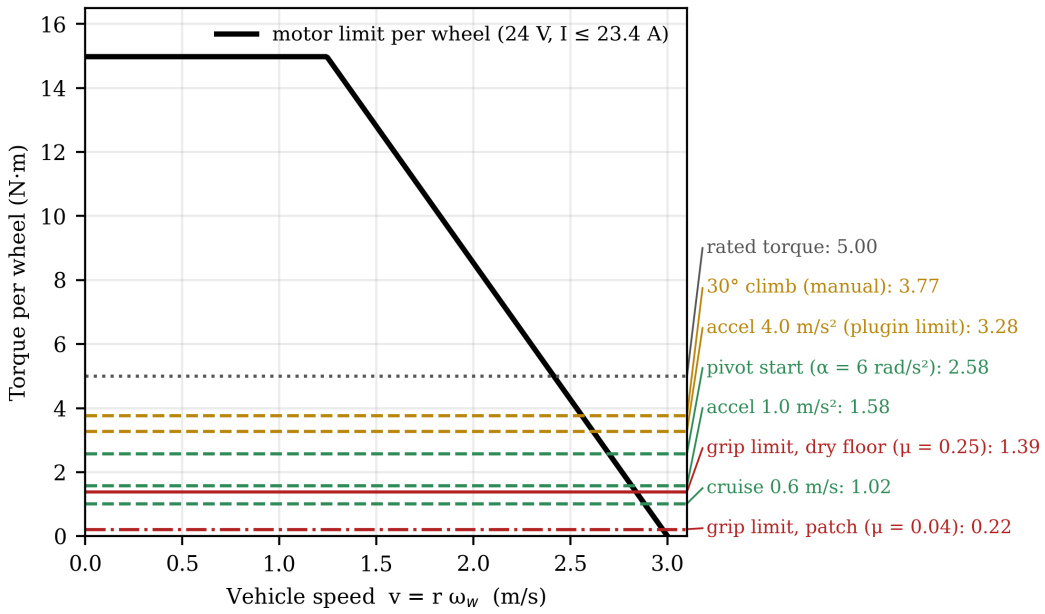


Fig. 4: Torque–speed limit of one wheel motor (current-limited, then voltage-limited) with the operating points of the design cases.

B. Worked example: cruise at 0.6 m/s

With a = 0 and θ = 0, Eq. (2) gives the force; each of the four wheels carries a quarter of it, the wheel speed is v / r, and Eqs. (5) and (6) give the torque and the current:

$$F = 0.03 \cdot 28.3 \cdot 9.81 + \frac{1}{2} \cdot 1.2 \cdot 0.1925 \cdot 0.6^2 = 8.33 + 0.04 = 8.37\ \text{N}, \qquad \omega_w = \frac{0.6}{0.08} = 7.5\ \text{rad/s} \tag{17}$$

$$\tau_w = \frac{8.37}{4} \cdot 0.08 + 0.1 + 0.1 \cdot 7.5 = 0.167 + 0.1 + 0.75 = 1.02\ \text{N} \cdot \text{m}, \qquad I = \frac{1.02}{0.64} = 1.6\ \text{A} \tag{18}$$

The cruise torque is 20 % of the rated 5.0 N·m. The joint damping term (0.75 N·m) is the largest part, because the URDF joints carry a damping of 0.1 N·m·s/rad.

C. The motor model in the simulation

The motors, the gearbox and the battery are part of the running simulation as the `motor_model` node, which sits between slip_control and the Gazebo wheels (Fig. 13). The motors and gearboxes were added to the robot model as visual bodies (one red 150 W motor and one grey gearbox housing at each wheel, without mass or collision); Figure 5 shows them in Gazebo with a translucent chassis, next to the live values that the node publishes while it cruises at 0.6 m/s: the torque, current and power of the motors, the battery voltage and charge, the traction demand against the grip of the dry floor and of a patch, and the torque-limit flag. The values on the right are the logged `/motor/*` topics of the run of Section 12-B and agree with the calculation above (1.02 N·m, 1.6 A). The motor and gearbox bodies are only visual: the wheels are still moved by the DiffDrive plugin, and the node calculates and limits the torque (Section 17); the battery has no body.

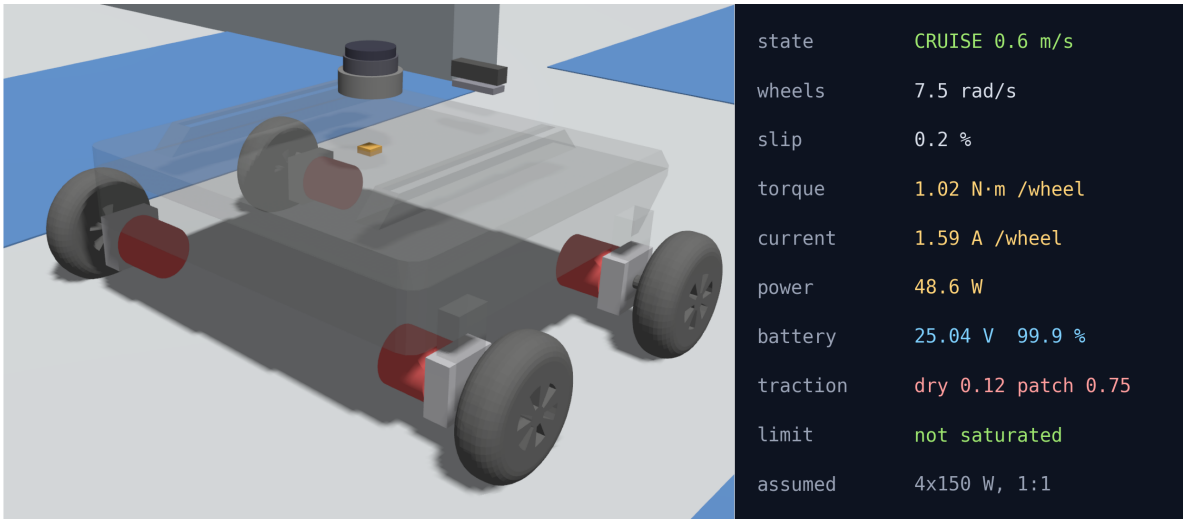


Fig. 5: The motors in Gazebo (left: red hub motors, grey gearboxes; the chassis is drawn translucent in this picture only) and the live motor values of motor_model during a 0.6 m/s cruise (right; logged topics, not a Gazebo window).

5. Torque and Force for Driving

Because the project studies slip on a flat floor rather than slopes, the driving cases are speed changes and turns. The table gives, per wheel, the torque from equations (2)–(8) for the simulated mass of 28.3 kg. The last two columns are the traction demand: the force demanded divided by the grip limit μ m g of Section 6 (a value above 1 means that the wheels must slip).

Table 5: Torque, current and traction demand per wheel for the design cases (m = 28.3 kg; the linear cases are taken at v = 0.6 m/s).

Case	**Force F [N]**	**τ per wheel [N·m]**	**Current [A]**	**τ / τ_{rated}**	**Demand, dry floor**	**Demand, patch**
Cruise, 0.6 m/s	8.4	1.02	1.6	20 %	0.12	0.75
Start, a = 1.0 m/s² (Nav2)	36.7	1.58	2.5	32 %	0.53	3.30
Brake, a = −1.5 m/s² (Nav2)	−34.1	0.17	0.3	3 %	0.49	3.07
Start, a = 4.0 m/s² (plugin limit)	121.6	3.28	5.1	66 %	1.75	10.95
30° climb at 0.6 m/s (manual check)	146.1	3.77	5.9	75 %	2.10	13.15
Pivot turn, ω = 1 rad/s steady	M = 22.5 N·m	1.95	3.0	39 %	–	–
Pivot start, α = 6 rad/s² (at ω = 1 rad/s)	M = 32.1 N·m	2.58	4.0	52 %	–	–

In braking, the joint friction and damping (0.85 N·m per wheel at 7.5 rad/s) and the rolling resistance already give a deceleration of about 1.8 m/s², so the Nav2 deceleration of 1.5 m/s² needs almost no motor torque (0.17 N·m); the force at the ground is still 34 N, which is what the traction demand uses. **Result.** The motors are never the limit: even the plugin-limit start of 4 m/s² needs only 66 % of the rated torque and the 30° climb that the manual quotes (75 %) is within the motor's capacity, with the peak torque of 15 N·m about three times higher. The limit is the floor: on a patch the traction demand of cruising is 0.75 of the grip, and any acceleration or braking asked of Nav2 (1.0 and −1.5 m/s²) is 3 times what the patch can give. This is why slip appears on the patches although the motors have a large reserve.

6. Traction Limit and Wheel Slip

Assumptions of the traction model	
S1	Coulomb friction between tyre and floor: the largest force the four wheels can transmit is $F_{max} = \mu\, m\, g$.
S2	The weight is shared equally by the four wheels.
S3	Slip begins as soon as the force demand exceeds F_{max}; the friction then stays at μ m g (no stick–slip).
S4	The Gazebo wheels follow the velocity command, so the slip is produced by the contact model of the floor tiles, with μ taken from the world file.

From S1–S3 the largest acceleration the floor allows and the grip torque per wheel are:

$$F_{max} = \mu\, m\, g\,, \qquad a_{max} = \mu\, g\,, \qquad \tau_{grip} = \frac{\mu\, m\, g\, r}{4} \tag{19}$$

$$s = \frac{v_w - v_{ground}}{\max(|v_w|,\, 0.2)} \tag{20}$$

Calculation (m g = 28.3 · 9.81 = 277.6 N):

$$\text{dry}: \; F_{max} = 0.25 \cdot 277.6 = 69.4 \text{ N}, \quad a_{max} = 0.25 \cdot 9.81 = 2.45 \text{ m/s}^2, \quad \tau_{grip} = \frac{69.4 \cdot 0.08}{4} = 1.39 \text{ N} \cdot \text{m} \tag{21}$$

$$\text{patch}: \; F_{max} = 0.04 \cdot 277.6 = 11.1 \text{ N}, \quad a_{max} = 0.04 \cdot 9.81 = 0.39 \text{ m/s}^2, \quad \tau_{grip} = \frac{11.1 \cdot 0.08}{4} = 0.22 \text{ N} \cdot \text{m} \tag{22}$$

Table 6: Grip limits of the two floors (weight 277.6 N).

Floor	μ	F_{max} [N]	a_{max} [m/s²]	τ_{grip} per wheel [N·m]
Dry floor (tiles)	0.25	69.4	2.45	1.39
Low-friction patch	0.04	11.1	0.39	0.22

The comparison with Table 5 explains the behaviour. On the dry floor a cruise needs 0.12 of the grip and the Nav2 start 0.53, so the robot never slips. On a patch the same start would need 3.3 times the available force, and even the steady cruise needs 75 % of it, because the rolling resistance alone is about 8.4 N against 11.1 N available. A robot that holds the commanded wheel speed on a patch therefore spins its wheels and the wheel odometry over-reports (Section 8). The torque the motors would apply in that case is far above τ_{grip}: the grip, not the motor, sets the acceleration. This is the central result of the calculation and the reason for the slip controller.

7. Slip Detection and Correction

Assumptions of the slip controller	
C1	The IMU does not depend on the wheels, so its integrated forward acceleration is an independent estimate of the ground speed.
C2	The wheel speed is taken from /odom, which is the commanded wheel motion; slip is the difference between it and the estimate.
C3	The estimate may be pulled towards the wheel speed only when wheel and IMU agree about speed and acceleration.
C4	Detection uses hysteresis (enter 0.15 m/s, exit 0.06 m/s) so the flag does not chatter.

slip_control sits between Nav2 and the wheels. The command path is Nav2 → /cmd_vel → slip_control → /cmd_vel_safe → motor_model → /cmd_vel_motor → ros_gz_bridge → Gazebo DiffDrive. The EKF reads the corrected odometry /odom_corrected instead of /odom and fuses both forward and sideways speed. Detection runs at the IMU rate (100 Hz) and the command limiter at 50 Hz.

A. Estimating the true speed and detecting slip

The node removes gravity from the forward acceleration using the IMU orientation, integrates it into a ground-speed estimate and blends it with the wheel speed in a complementary filter. A zero-velocity update is applied only when the wheels, the accelerometer and the gyroscope are all quiet, and the estimate is kept below an energy bound.

$$v_{est} \leftarrow v_{est} + (a_f - a_b)\,dt \tag{23}$$

$$v_{est} \leftarrow v_{est} + \min(3\,G\,dt,\ 1)\,(v_w - v_{est}), \quad G = e^{-(d/0.05)^2}\, e^{-((a_w - a_f + a_b)/0.4)^2} \tag{24}$$

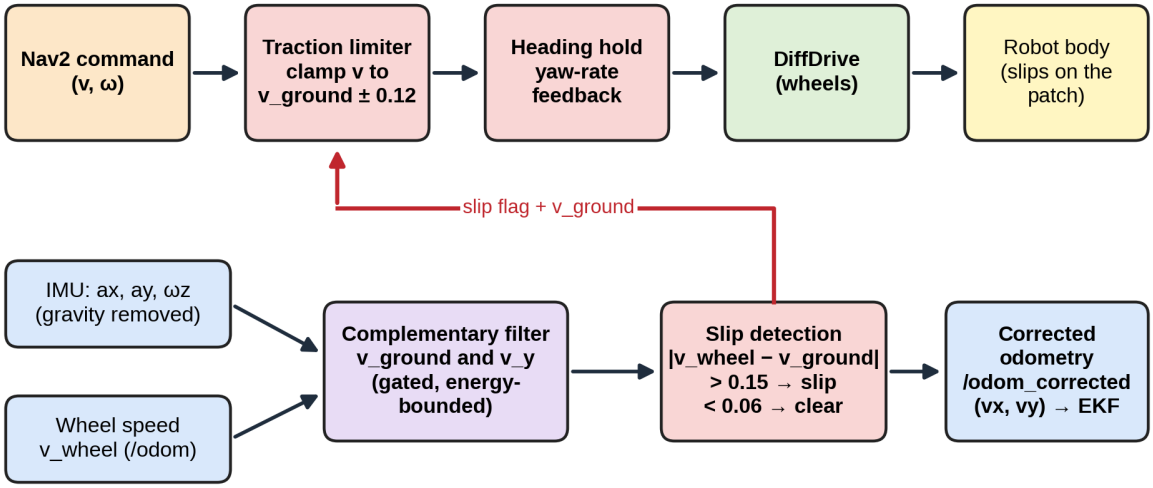
$$|v_{est}| \le v_{peak} + 0.05, \qquad v_{peak} = \max(|v_w|,\ v_{peak} - 0.3\,dt) \tag{25}$$

Table 7: Detection parameters (defaults of slip_control.py).

Parameter	Value	Purpose
Slip enter threshold	$\lvert v_w - v_{ground}\rvert$ > 0.15 m/s for 0.05 s	Declares slip; avoids false alarms from noise
Slip exit threshold	below 0.06 m/s	Clears the slip flag
Fusion gate	0.05 m/s and acceleration agreement	A spinning wheel cannot corrupt the estimate
Latch breaker	body steady 0.5 s while slip is flagged	Resets the estimate so slip does not stay on

Parameter	Value	Purpose
Energy bound	v_{peak} + 0.05 m/s, decay 0.3 m/s²	Stops integration drift from running away

B. The four correction actions



Gazebo wheels follow velocity commands rigidly, so the limiter keeps the commanded acceleration within what the floor can deliver.

Fig. 6: Control chain of slip_control: wheel speed and IMU give the slip decision, which drives the traction limiter; heading hold and odometry correction run continuously.

Table 8: The four correction actions.

Action	What it does	Reason
1. Traction limiter	While slipping and for 0.6 s after, the wheel command is clamped to v_{ground} ± 0.12 m/s and can never exceed the larger of the request and the current output; angular acceleration halved (6.0 to 3.0 rad/s²)	Like traction control in a car: less torque lets the wheels grip again
2. Heading hold	IMU yaw rate fed back (gain 0.4, limit ±0.6 rad/s)	Uneven grip makes the robot veer
3. Sideways slip	Lateral speed v_y integrated from the IMU lateral acceleration minus the turning term; pulled to zero when the tyres grip	Wheel odometry cannot see a sideways slide
4. Odometry correction	/odom_corrected carries the wheel speed with the slip removed (dead-bands 0.04 m/s for v_x, 0.03 m/s for v_y) and the sideways speed	The map pose stays close to the true pose

The limiter, as implemented, may only hold back an acceleration or release a brake: the target is also clamped between the request and the current output. It therefore acts during acceleration and braking on a patch, which is where the grip is exceeded (Table 5), and leaves a steady cruise unchanged; the traces of Section 12 show exactly this. The correction is switched off with the launch argument slip_correction:=false; the node then still estimates and publishes slip (/slip/active, /slip/ratio, /slip/traction_scale, /slip/ground_speed_est and the latched /slip/correction_enabled) but passes the command and the odometry through.

8. The Slip World in Gazebo

A second world, civil_world_slip.sdf, is generated from the civil world by generate_slip_world.py. It is the same facility with three flush low-friction patches of μ = 0.04, against μ = 0.25 for the dry tiles. The patches had to be flush with the floor: a first version with raised tiles stopped the robot at the 3 mm edge (problem 1, Section 14). The generator therefore removes the collision of the ground plane, lays floor tiles around holes of the patch size and fills the holes with the low-friction patches. The corridor patches lie between the corridor hub and the columns and cover the full corridor width, so that turns at the doors happen on dry floor (problem 13).

Table 9: Low-friction patches of the slip world (verified in civil_world_slip.sdf).

Patch	**Centre (x, y) [m]**	**Size (x × y) [m]**	**μ**
Conference Hall doorway (north)	(0.0, 2.3)	1.2 × 1.8	0.04
East corridor	(1.1, 0.0)	1.0 × 2.4	0.04
West corridor	(−1.1, 0.0)	1.0 × 2.4	0.04

With the correction off the wheels turn faster than the body moves (in a Ground Station capture: wheels 0.48 m/s, ground 0.40 m/s, slip 16 %), the odometry counts distance that was never travelled, and the EKF and AMCL believe the robot is farther along than it is. A first straight-line test on an earlier, slipperier floor (μ = 0.02) gave a position error of +1.12 m with the correction off, against +0.04 m with it on.



Fig. 7: The slip world from above (Gazebo camera render): the three blue areas are the low-friction patches.

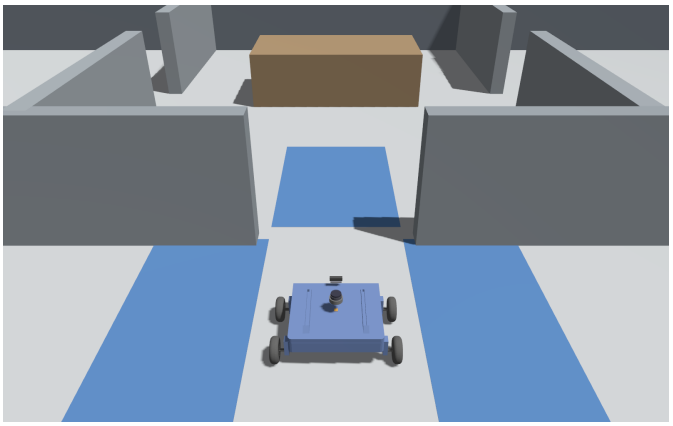


Fig. 8: The robot at the corridor hub, with the corridor patches ahead (Gazebo camera render).

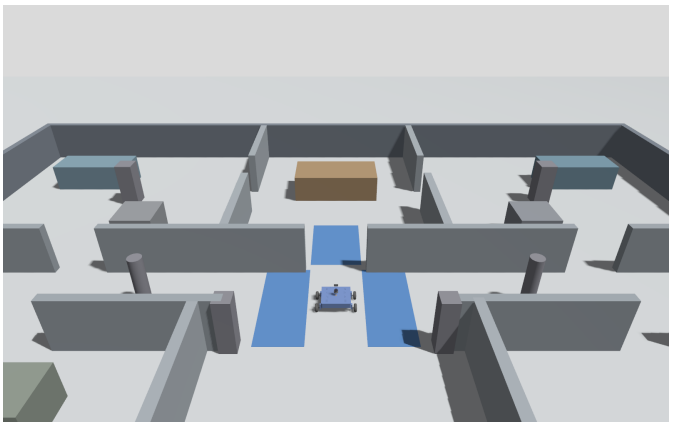


Fig. 9: The robot in the slip world, low view (Gazebo camera render).

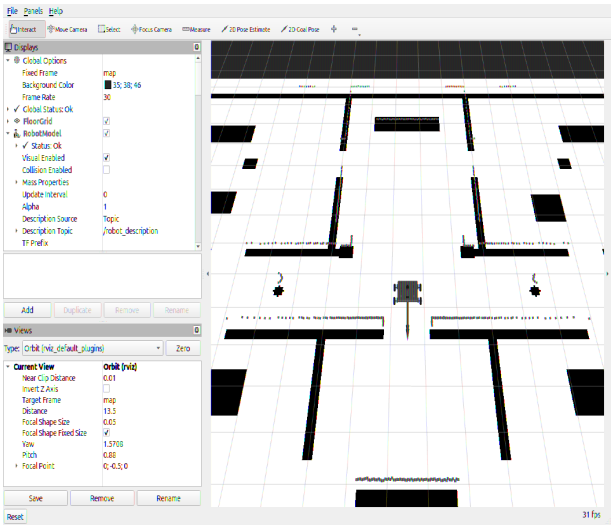


Fig. 10: RViz2 view of the slip world: map, scan and robot model. The patches carry no map marking, so slip must be detected from motion.

9. Ground Station and Direct Routing

Slip panel. The Ground Station shows a Wheel Slip Monitor: a slip percentage bar, wheel speed against ground speed, the traction scale, the surface under the robot (“normal floor” or “LOW-FRICTION” with the zone name) and the correction mode. The label reads “GRIP OK”, “SLIP DETECTED – traction control + odometry correction” or, with the correction off, “SLIP DETECTED – correction OFF (monitor only)”. The three patches are drawn on the map as blue hatched boxes. The first version said “correction ACTIVE” even when it was off; the latched topic /slip/correction_enabled fixed this.



Fig. 11: Ground Station on a patch: slip detected, wheels 0.35 m/s, ground 0.19 m/s, traction reduced to 64 %.

Click-to-navigate by the shortest path. The earlier version split each click into door-aligned waypoints, which forced detours and let an old route ignore a new click. The Ground Station now sends one NavigateToPose goal and the Smac 2D planner finds the shortest path through the doorways. Each click increments route_id; an older route notices that it was superseded and stops without cancelling the new goal. The clicked point is moved to the nearest cell at least 0.55 m from obstacles, and a goal the robot has stayed within 0.30 m of for 6 s counts as reached. In Nav2 cost_travel_multiplier is 1.0 and yaw_goal_tolerance 3.14 rad, so the planner minimises distance and the robot does not spin to a final heading.

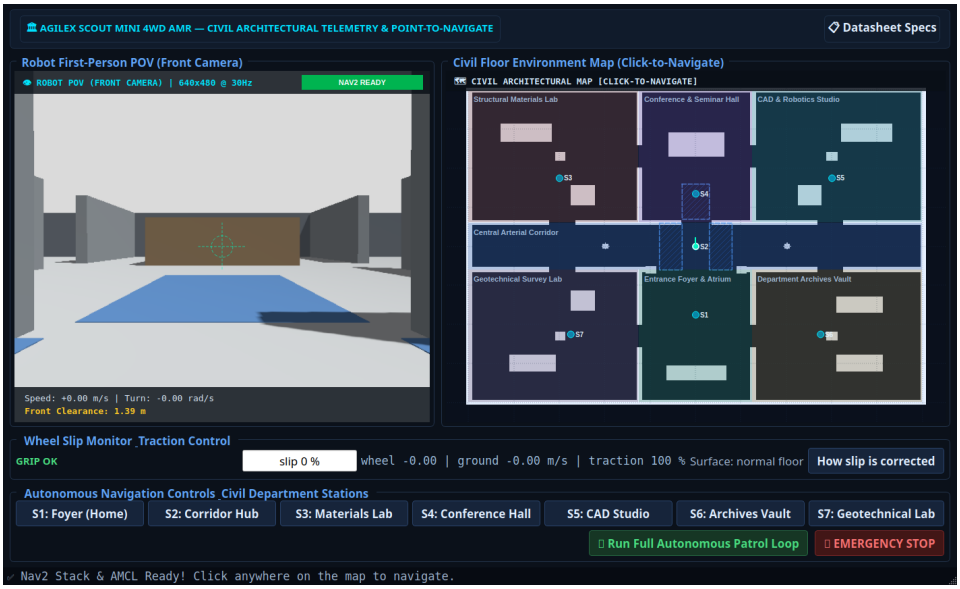


Fig. 12: Ground Station on the normal floor: camera view, map with the six rooms and the slip panel (GRIP OK).

10. Software Changes

The tables below list the data flow and the nodes; Figure 13 at the end of the section shows the running system. New or changed parts are the slip_control node, the motor_model node, the corrected odometry and the remapped bridge input.

Table 10: Data flow of the final system.

Step	From → To	Topic	What it does
1	Gazebo → ros_gz_bridge	/scan, /imu, /odom, /clock, /camera/image_raw	Sensors and wheel odometry enter ROS 2
2	Bridge → slip_control, EKF, AMCL	/imu, /odom, /scan	slip_control receives raw odometry and IMU
3	slip_control → EKF	/odom_corrected (vx, vy)	Slip-corrected odometry replaces raw /odom
4	Nav2 → slip_control	/cmd_vel	Command from the velocity smoother and collision monitor
5	slip_control → motor_model	/cmd_vel_safe	Traction limiter and heading hold act while slip is detected
6	motor_model → bridge	/cmd_vel_motor	Command after the torque-limit check; the bridge forwards it to DiffDrive
7	motor_model → Ground Station and logs	/motor/torque, current, power, battery_voltage, battery_soc, saturated, traction_dry, traction_patch	Calculated drive quantities
8	slip_control → Ground Station	/slip/active, ratio, traction_scale, ground_speed_est, correction_enabled	Slip panel

Table 11: Nodes and programs of the system.

Node or program	Package or file	Role
ros_gz_bridge	ros_gz_bridge	Bridges /clock, /scan, /imu, /odom, /camera/image_raw, /joint_states; subscribes to /cmd_vel_motor
robot_state_publisher	robot_state_publisher	URDF kinematic tree and wheel joint transforms
ekf_node	robot_localization	Fuses corrected wheel speed and IMU yaw rate; odom → base_footprint
map_server, amcl, planner, controller, smoother, bt_navigator	Nav2	Map, localisation, Smac 2D planner, regulated pure pursuit, velocity smoother
slip_control	assem2_robot/scripts	Slip estimation, traction limiter, heading hold, /odom_corrected
motor_model	assem2_robot/scripts (new)	Torque, current, power, battery, grip-limit comparison; torque limit
civil_interactive_nav.py	assem2_robot/scripts	PyQt5 Ground Station with slip panel
civil_nav2_mission.py, nav_tour_test.py	assem2_robot/scripts	20-leg patrol with ground-truth verification; route-efficiency tour
generate_civil_world.py, generate_slip_world.py	assem2_robot/scripts	Generate the facility world and the slip world

The launch file civil_simulation.launch.py was changed to remap the bridge input to /cmd_vel_motor, to start motor_model, and to take the arguments headless and use_rviz so that tests can run without any window.

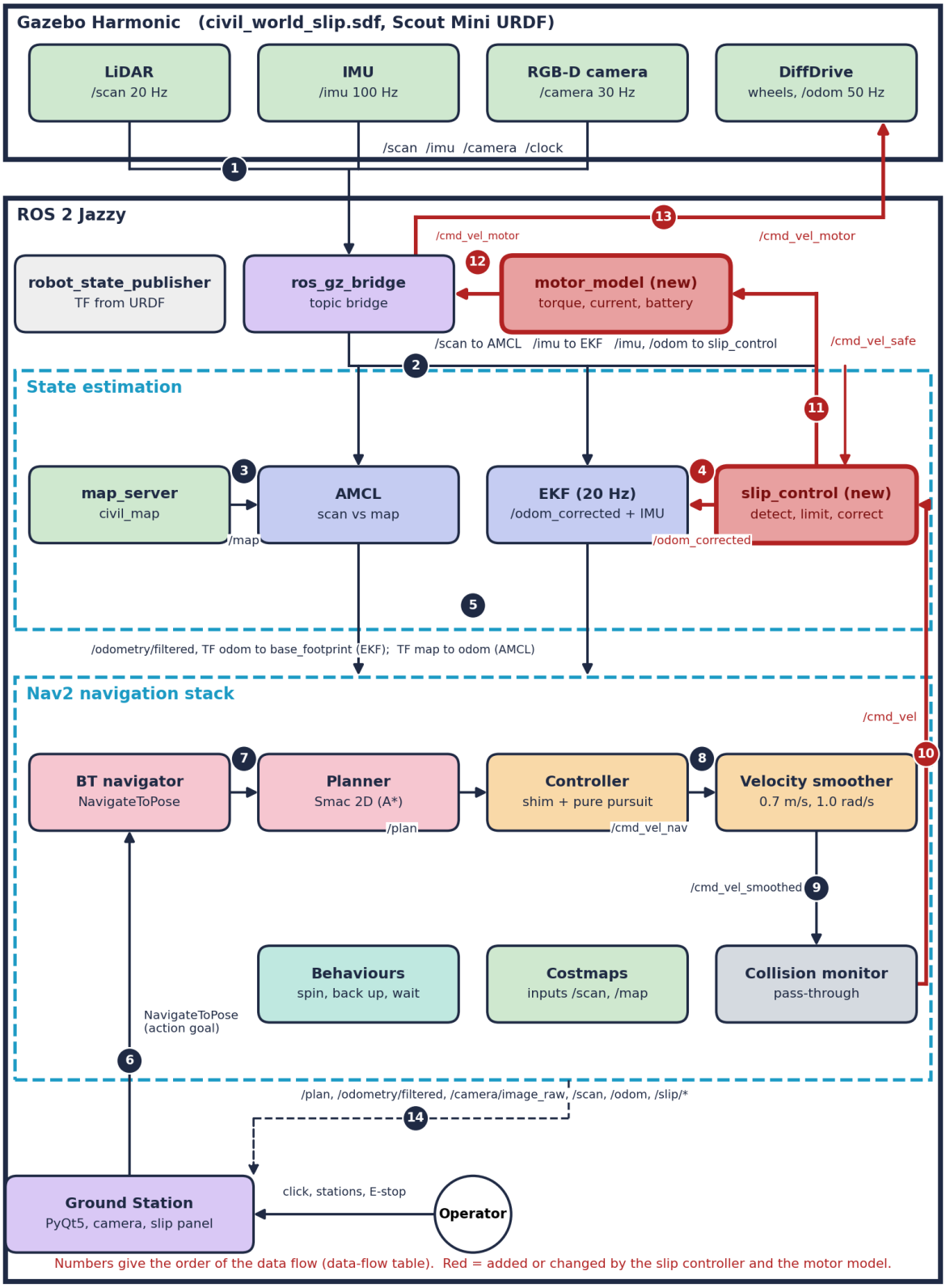


Fig. 13: Architecture with slip control and the motor model. The command path ends at the Gazebo DiffDrive plugin through /cmd_vel_motor.

11. Test Method

Patrol. The patrol has 20 goals (legs) through the Conference Hall, the CAD Studio and the Materials Lab and back to the corridor hub; four legs are stations where the robot waits 2 s. Each run starts from a freshly launched headless simulation with AMCL initialised at (0, 0). A leg succeeds when Nav2 reports success and both the AMCL pose and the Gazebo ground-truth pose are within 0.45 m of the goal; at station legs the heading must also be within 40°. A failed leg is retried up to three times; a leg that fails all attempts ends the run; a leg times out after 120 s. Three conditions were run, three times each: the normal floor, the slip floor with the correction on, and the slip floor with the correction off.

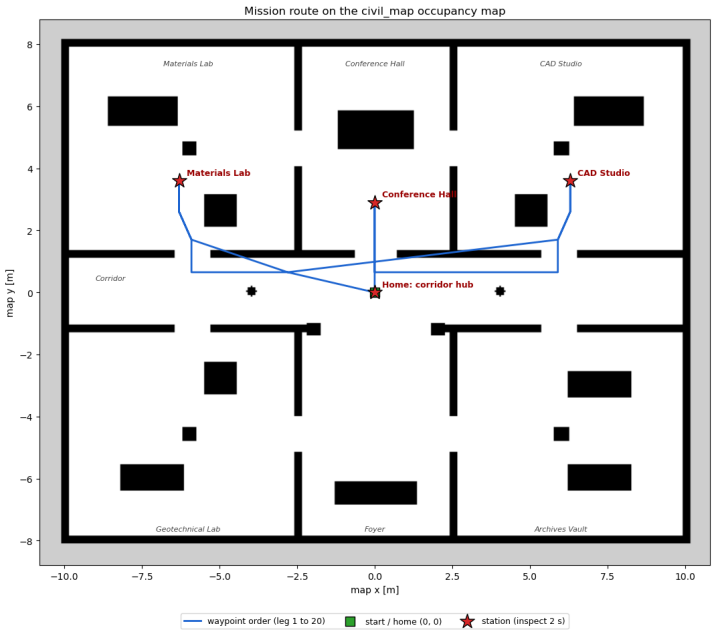


Fig. 14: The 20-leg patrol on the occupancy map. The blue line joins the waypoints in order; it is not the measured path.

Route-efficiency tour. Seven goals were sent as single direct goals. The driven distance is the sum of displacements of the map → base_footprint transform at about 5 Hz, compared with the length of a grid A* path with 0.30 m clearance.

Motor readings. The motor_model topics were recorded on the simulation at 0.5 m/s straight driving, at a start and in a 1 rad/s spin, and compared with the hand calculation of Section 5. **Patch crossing.** The robot was driven straight at 0.6 m/s across the slip floor with the correction on and off (22 s each, in a fresh simulation, no EKF), and wheel speed, the slip estimate, torque and battery were logged.

12. Results

A. Motor model: calculated against measured

Table 12: Calculated and measured drive quantities.

Case	Calculated	Measured (Gazebo, headless)	Note
Straight at 0.5 m/s: torque per wheel	0.89 N·m	0.89 N·m	equations (2), (5)
Straight at 0.5 m/s: current / power	1.4 A, 39 W	1.4 A, 39 W	electrical input power incl. 12 W idle
Start (a = 4 m/s², plugin limit, at 0.5 m/s)	3.16 N·m	2.7–3.2 N·m peak	peak depends on sampling

Case	Calculated	Measured (Gazebo, headless)	Note
Spin at 1 rad/s: torque per wheel / power	±1.95 N·m, 64 W	±2.0 N·m, 70 W	equations (3), (4)
Battery voltage: rest / 0.5 m/s / spinning	–	25.16 / 25.08 / 24.98 V	equation (9)
State of charge after about 25 s	–	100 → 99.95 %	15 Ah pack
Torque-limit flag (saturated)	false	false	demand far below 15 N·m

The calculation and the simulation agree at cruise and during the spin to within 0.05 N·m (the spin power differs by 6 W, 9 %); the start differs because the peak torque depends on how the 4 m/s² plugin acceleration is sampled. The battery voltage sags by 0.08 V at cruise and by 0.18 V while spinning, of the same order as $R_{int} I_{bat}$ for R_{int} = 0.08 Ω and battery currents of about 1.5–3 A.

B. Crossing a low-friction patch, correction on and off

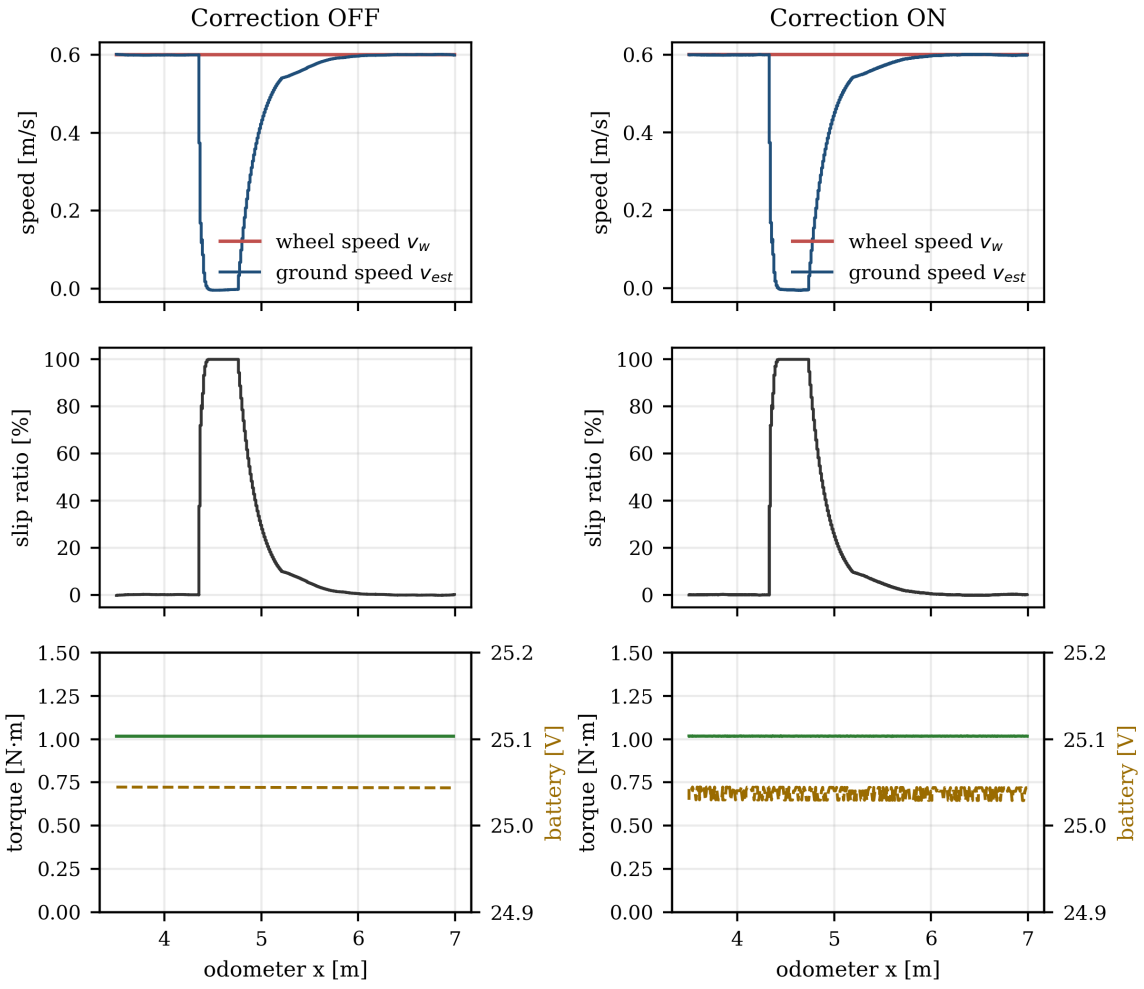


Fig. 15: Straight drive at 0.6 m/s across a patch, correction OFF (left) and ON (right), against the odometer position. The first row shows wheel speed and the IMU ground-speed estimate, the second the slip ratio, the third torque per wheel and battery voltage.

In both runs the estimator flags slip when the robot enters the patch: the ground-speed estimate falls from 0.6 m/s to about zero and the slip ratio rises to 100 % for about 1.3–1.4 s, then recovers as the estimate returns to the wheel speed. The wheel speed, the commanded speed and the torque (1.02 N·m) stay constant in both runs and the traction scale stays 1.0, because the robot is cruising

at a steady speed and the limiter only holds back accelerations (Section 7). The two traces are therefore almost identical, and this test does **not** show a benefit of the correction. The odometer position is wheel odometry and no ground-truth pose was logged here, so the true motion cannot be read from these plots; the effect of the correction is visible only in the patrol results below, where braking and starting happen on the patches.

C. Normal floor (20-leg patrol)

All three planned runs completed all 20 legs without any retry, in 198–251 s. Ground-truth arrival errors averaged 0.23 m and never exceeded 0.37 m (limit 0.45 m); the mean AMCL error was 0.18 m. An additional earlier run with the same script also completed 20 of 20 legs (203 s, worst error 0.29 m).

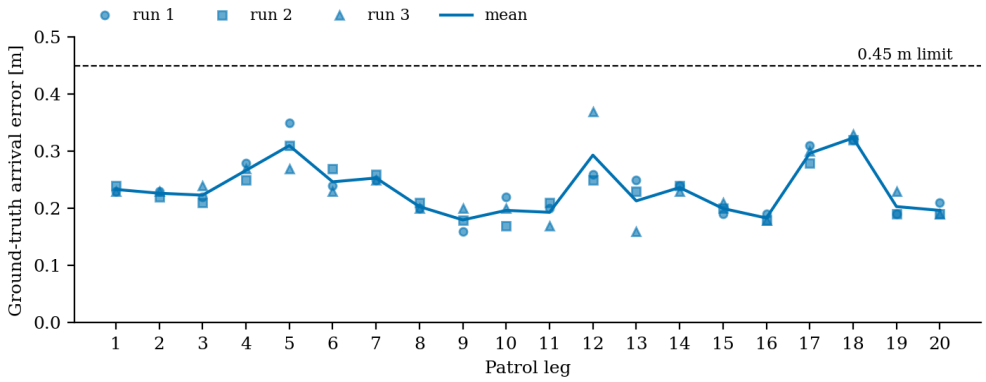


Fig. 16: Arrival error at each leg of the three normal-floor runs (dots) and their mean (line).

Table 13: Every run made with the final navigation configuration.

Condition	Runs made	Completed 20/20	Not completed	Wall time of completed runs [s]
Normal floor	4 (3 planned + 1 earlier)	4	–	198, 203, 210, 251
Slip floor, correction ON	4 (3 planned + 1 earlier)	3	1 stopped at leg 9 (CAD Studio station): heading error 45–46° against the 40° limit on all three attempts, position error 0.14 m	300, 335, 402
Slip floor, correction OFF	3	2	1 stopped at leg 4 after 44 s: the robot overshot the corridor hub by 0.61, 0.51 and 0.48 m (limit 0.45 m)	213, 243

D. Slip floor: correction ON compared with OFF

Table 14: Correction OFF against ON in the planned nine-run set.

Measure	Correction OFF	Correction ON
Wheel command on a patch	Passes unchanged; wheels spin	Clamped to ground speed ± 0.12 m/s; traction scale reduced (64 % in Fig. 11)
Odometry	Raw wheel odometry over-reports	Slip-corrected /odom_corrected (vx and vy)
Planned runs completed	2 of 3 (legs reached: 3, 20, 20)	3 of 3 (legs reached: 20, 20, 20)
Mean / worst arrival error (truth)	0.28 / 0.45 m	0.22 / 0.33 m

Measure	Correction OFF	Correction ON
Leg 4 (corridor hub), first arrival	Overshot the 0.45 m limit in 3 of 3 runs (0.46–0.61 m)	Within the limit in 3 of 3 runs
Retries	5 (all at leg 4)	5 (all at leg 12, aborted by Nav2)
Wall time of completed runs [s]	213, 243	300, 335, 402 (mean 346, against 220 on the normal floor)

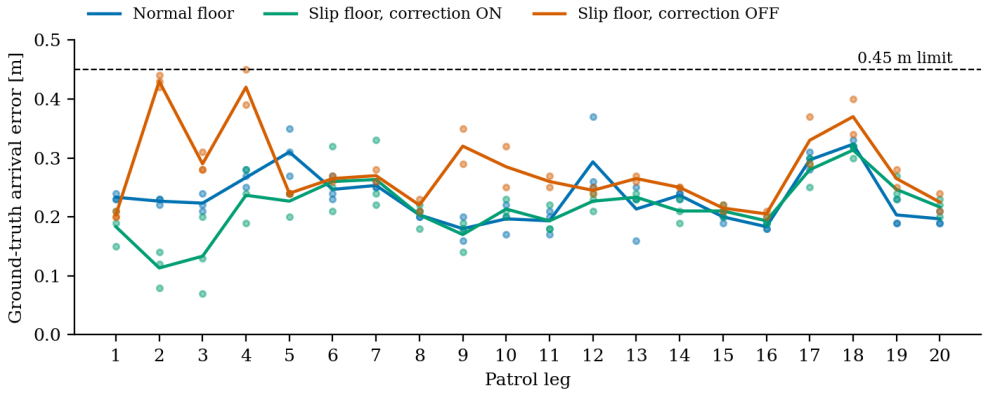


Fig. 17: Ground-truth arrival error at each leg in the three conditions (lines: mean; dots: single runs).

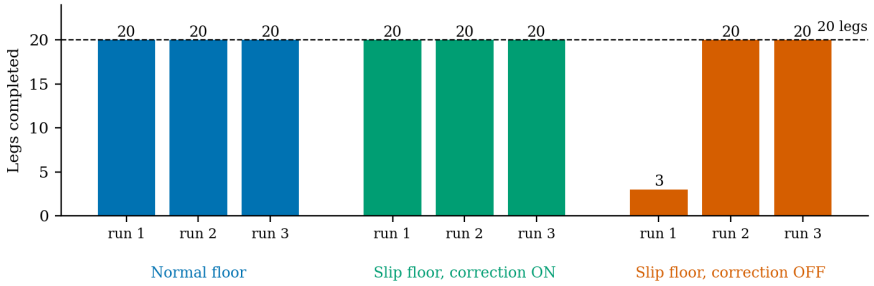


Fig. 18: Legs completed (of 20) in each planned patrol run.

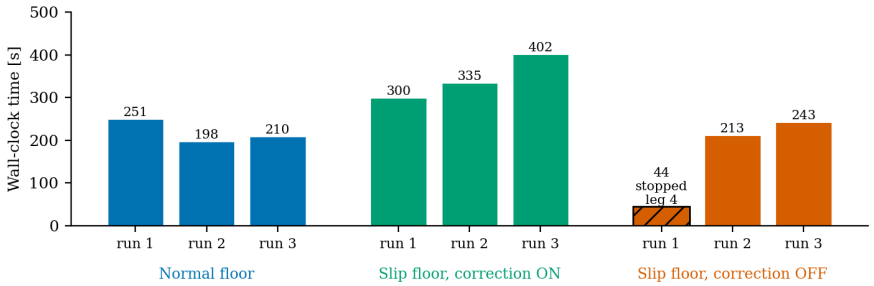


Fig. 19: Wall-clock time of the nine planned patrol runs; the hatched bar is the run with the correction off that stopped at leg 4.

With the correction on, all three planned runs completed and the worst arrival error was lower (0.33 m against 0.45 m). The clearest difference is leg 4: with the correction off the robot slid past the corridor hub in all three runs after leaving the Conference Hall doorway patch, which is a braking case, where Table 5 gives a demand of 3 times the grip. The correction costs time: the completed runs took 300–402 s against 198–251 s on the normal floor, and leg 12 was aborted by Nav2 once or twice in every run before it succeeded on a retry; the cause was not investigated. With three runs per condition the difference in completion (3 of 3 against 2 of 3) is not statistically significant; the leg-4 overshoot, in all three uncorrected runs and in none of the others,

is the more reliable signal.

E. Route efficiency

Table 15: Route-efficiency tour on the normal floor (all goals reached, status 4), with the totals of the slip-floor tour (correction on). On the slip floor the legs differ by −3…+6 %, and S7 took 67 s against 33 s.

Goal	Driven [m]	Optimal [m]	Excess	Time [s]	Arrival error [m]
S3 Materials Lab	8.9	8.7	+2 %	19	0.19
S5 CAD Studio	13.3	12.8	+4 %	28	0.18
S7 Geotechnical Lab	16.3	15.9	+3 %	33	0.18
S6 Archives Vault	10.3	10.4	−1 %	23	0.18
S4 Conference Hall	10.8	10.8	+0 %	27	0.23
S1 Foyer	6.3	6.0	+4 %	18	0.18
S2 Hub	3.4	3.5	−2 %	12	0.19
Total, normal floor	**69.3**	**68.1**	**+1.8 %**		
Total, slip floor, correction on	**69.5**	**68.0**	**+2.2 %**		

Every goal was reached on both floors and the driven distance was between −3 % and +6 % of the grid reference per leg. Negative values are possible because the reference keeps 0.30 m from obstacles and the measured distance comes from a sampled, filtered pose. Route length is therefore not the reason for the longer slip-floor patrol times.

13. Unit Tests

The motor model has six unit tests (pytest, no simulator needed): `python3 -m pytest src/assem2_robot/tests -q`. All six pass. They check that the calculated torque, current and grip limits match the values used in this report.

Table 16: Unit tests of motor_model.py.

Test	What it checks	Expected
test_stall_torque_matches_current_limit	τ_{max} at standstill = $K_t I_{peak}$	14.98 ≈ 15.0 N·m
test_no_load_speed_is_voltage_limited	Available torque at V / K_e = 37.5 rad/s	0 N·m
test_torque_falls_with_speed_above_the_current_limit_knee	Torque lower above the knee at 15.6 rad/s	monotone decrease
test_cruise_and_acceleration_torque_match_table	Cruise 1.02 N·m, start 1.58 N·m (Table 5)	within tolerance
test_grip_limits_are_far_below_motor_torque	τ_{grip} = 1.39 N·m dry, 0.22 N·m patch	well below 15 N·m
test_gearbox_ratio_scales_torque_and_current	Gear ratio N scales torque and current	proportional

14. Problems Solved

Problems 1–16 were found while adding and testing slip; 17 and 18 while re-measuring; 19–21 while adding the motor layer.

Table 17: Problems met in the second review.

#	**Problem**	**Root cause**	**Fix**
1	Robot stopped at the edge of a patch	Raised tiles: a low-grip robot cannot climb the edge; odometry keeps counting	Flush floor tiles with patch holes
2	Edits of slip_control.py had no effect	CMake installs a copy of the script	colcon build --symlink-install after every edit
3	Slip reported while stopped; correction stuck on	Estimate biased low after a stop	Gated fusion, zero-velocity update only when quiet, latch breaker, energy bound
4	Wheel speed ran away to 1.44 m/s	clamp(cmd, v_est − m, v_est + m) could raise the command above the request	Target bounded between the request and the current output
5	Robot slid sideways with no change in odometry	Wheel odometry cannot see lateral slip	Lateral speed estimated from the IMU and fused in the EKF
6	Believed position drifted from the truth	Wheel odometry over-reported on slip	Slip-corrected odometry in the EKF
7	The argument world:= was ignored	Fixed world path in the launch file	Launch file uses the world argument
8	Leftover Gazebo/RViz processes corrupted TF	Processes of earlier runs still running	Each run in its own process group, stopped by PID
9	Clicking a goal sent the robot elsewhere; several clicks needed	Door-aligned waypoints; new click ignored during a route	Single direct goal with route_id pre-emption
10	Screenshots were black	Wayland blocks screen capture	Qt off-screen render; Gazebo camera sensors; xwd for RViz2
11	Panel said “correction ACTIVE” with correction off	GUI could not tell the mode	Latched /slip/correction_enabled
12	Identical runs gave different results	Contact physics is not repeatable	Not fixed; all runs are reported
13	Robot slid sideways into a corridor post	Heading error carried onto ice; 0.02 m clearance	Corridor patches moved to the dry hub side and made full width
14	A second test simulation changed the open one	Launch file forced ROS_DOMAIN_ID 19	Launch keeps an exported ROS_DOMAIN_ID
15	Patrol stalled for 120 s at the east door	Forced goal heading made the robot rotate in place; stale installed script	Forced heading removed, script rebuilt
16	Early theory (uneven grip) did not explain slides	Real cause was heading error on ice (13)	Traced with recordings of true pose
17	civil_nav2_mission ran an old copy of the patrol script (different leg list)	Installed copy not rebuilt	Symlink install; compare installed file before runs
18	A run stopped at a station with the robot 0.14 m from goal	Station heading ≤ 40° checked but yaw tolerance is 3.14 rad	Open: tighter tolerance only on station goals
19	RViz2 and Gazebo windows opened during tests and showed errors	Test launches started the full GUI	Launch arguments headless:=true and use_rviz:=false
20	Wrong motor power (250 W) in an early draft	Value taken from a reseller page	Replaced by the 150 W of the official manual

#	Problem	Root cause	Fix
21	Unit test expected 15.0 N·m peak torque	0.64 × 23.4 = 14.976 N·m	Test expects 14.976 (≈ 15.0)

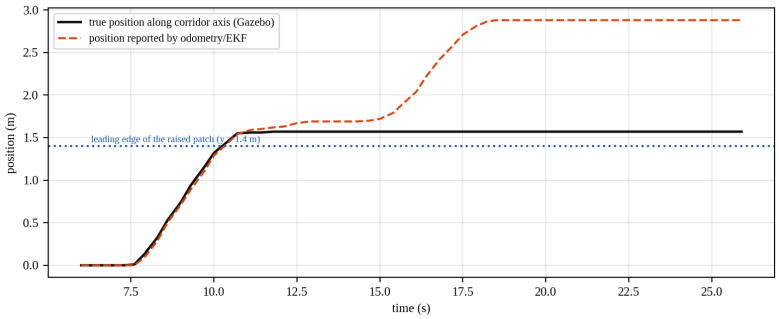


Fig. 20: Problem 1: with the raised patch edge the true robot (black) stops at y = 1.57 m against the edge, while its odometry (orange, dashed) keeps counting and drifts to 2.88 m.

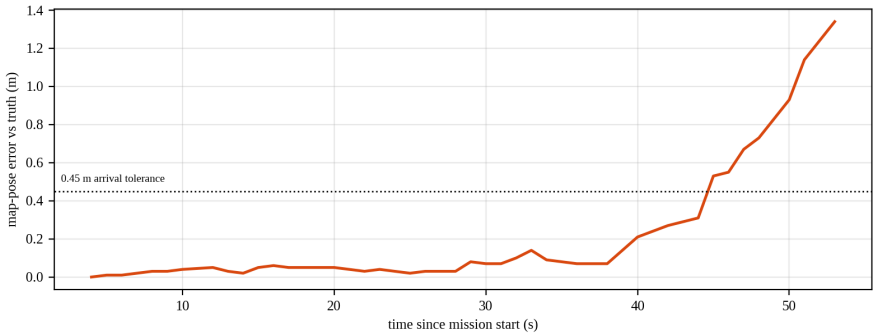


Fig. 21: Problem 6: map-pose error in an early run before the correction was added in its final form; it passes the 0.45 m tolerance at about 45 s.

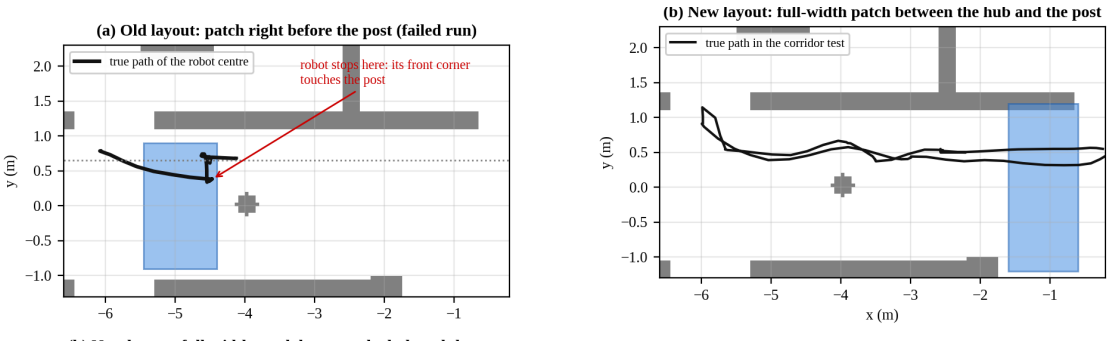


Fig. 22: Problem 13: old layout (a), the robot stops with a front corner on the post; final layout (b), the patch lies between the hub and the post.

15. Algorithms Used

Table 18: Algorithms of the second review.

Algorithm	Where	Purpose
DC motor, gearbox and battery calculation (eqs. 1–18)	motor_model.py	Torque, current, power, battery, torque limit
Traction (grip) comparison, eqs. 19–20	motor_model.py	Flags force demand above μ m g
Complementary-filter ground-speed estimate	slip_control.py	Independent speed from the IMU
Hysteresis slip detection with debounce	slip_control.py	Slip flag

Algorithm	Where	Purpose
Traction limiter and heading hold (50 Hz)	slip_control.py	Command limiting and yaw-rate feedback
Sideways slip estimation; odometry correction	slip_control.py	Corrected pose for the EKF
Click-to-navigate with pre-emption	civil_interactive_nav.py	One click, one goal
Patrol leg with closed-loop verification	civil_nav2_mission.py	Ground-truth check of each leg
Route-efficiency measurement	nav_tour_test.py	Driven versus optimal distance
Smac 2D A*, regulated pure pursuit, AMCL, EKF	Nav2, robot_localization	Unchanged from the first review

Algorithm 1. Motor model step (per command, 50 Hz)

```
v, w <- command from /cmd_vel_safe;  a <- (v - v_prev)/dt;  alpha <- (w - w_prev)/dt
F  <- m a + c_rr m g sgn(v) + 0.5 rho CdA v|v|                  # eq. 2 (flat floor)
M  <- Iz alpha + mu_t m g (L/2) sgn(w)                          # eq. 3
F_R, F_L <- F/2 + M/B, F/2 - M/B                                # eq. 4
for each side:  tau <- F_side r/2 + tau_f + b omega_w           # eq. 5
                I <- tau/(N eta Kt);  V <- Ke N omega_w + R I   # eqs. 6, 7
                tau_max <- Kt min(I_peak, (V_bat - Ke N|omega_w|)/R) N eta     # eq. 8
if enforce_limits and tau > tau_max: reduce the acceleration until tau <= tau_max; saturated <- True
P <- sum(V I) + P0;  I_bat <- P / V_bat;  V_bat <- V_oc(SOC) - R_int I_bat;  SOC <- SOC - I_bat dt / (15 Ah)
traction_dry <- F / (0.25 m g);  traction_patch <- F / (0.04 m g)                 # eq. 19
publish /motor/*, and /cmd_vel_motor (the command after the check)
```

Algorithm 2. Slip detection and correction (slip_control)

```
a   <- forward IMU acceleration - gravity leak;   a_f <- a_f + 0.3 (a - a_f)
if quiet for 0.3 s: v_est <- v_w; update bias                  # zero-velocity update
else: v_est <- v_est + (a_f - a_b) dt;  if not slip: pull v_est to v_w with gate G      # eq. 23, 24
v_est <- clamp(v_est, -(v_peak + 0.05), v_peak + 0.05)                                   # eq. 25
slip_speed <- v_w - v_est;  ratio <- clamp(slip_speed / max(|v_w|, 0.2), -1, 1)
slip <- True if |slip_speed| > 0.15 for 0.05 s;  slip <- False if |slip_speed| < 0.06
limiting <- slip or (now - t_clear) < 0.6
target_v <- c_v;  if limiting: target_v <- clamp(c_v, v_est - 0.12, v_est + 0.12)
                               target_v <- clamp(target_v, min(c_v, out_v), max(c_v, out_v))
out_v <- slew(out_v, target_v, 4.0 m/s^2);  scale <- |out_v| / |c_v|
out_w <- slew(c_w + clamp(0.4 (c_w - w_z), -0.6, 0.6), 6.0 (3.0 if limiting) rad/s^2)
```

Algorithm 3. Click-to-navigate with pre-emption (Ground Station)

```
on map click (x, y): route_id <- route_id + 1; my_id <- route_id; cancel the active goal
    (gx, gy) <- nearest free cell >= 0.55 m from any obstacle;  send ONE NavigateToPose goal
    while goal active and my_id == route_id:
        if distance(robot, goal) < 0.30 m for 6 s: cancel goal; count as reached
    if my_id != route_id: stop quietly
```

16. How to Run

All commands are run on Ubuntu 24.04 with ROS 2 Jazzy and Gazebo Harmonic. The launch files set ROS_DOMAIN_ID 19 (an exported value is respected) and local-only discovery.

```
cd ~/agilex && colcon build --symlink-install      # rebuild after every edit of a script
source install/setup.bash
# normal floor, full system with Ground Station
ros2 launch assem2_robot civil_simulation.launch.py use_nav:=true use_gui:=true
# slip world, correction on (default) or off
ros2 launch assem2_robot civil_simulation.launch.py use_nav:=true use_gui:=true \
     world:=$HOME/agilex/install/assem2_robot/share/assem2_robot/worlds/civil_world_slip.sdf slip_correction:=false
# headless, for tests and logging (no Gazebo or RViz window)
ros2 launch assem2_robot civil_simulation.launch.py headless:=true use_rviz:=false
# motor model: watch the calculated quantities
ros2 topic echo /motor/torque ; ros2 topic echo /motor/battery_voltage
# patrol with verification, and the route tour
ros2 run assem2_robot civil_nav2_mission ; python3 src/assem2_robot/scripts/nav_tour_test.py
# unit tests of the motor model
python3 -m pytest src/assem2_robot/tests -q
```

17. Limitations and Future Enhancements

- **The motor layer does not move the robot.** Gazebo DiffDrive follows the velocity command, so motor_model reports the torque and current the command would need and limits the acceleration only if a motor reached its limit, which never happened in the tests. It does not drive the wheels by force, so slip comes from the contact model and the friction of the world file, not from the motor torque. A torque-driven drive is a future step.
- **Several motor values are assumed** (K_t, R, the 1 : 1 gear ratio, battery resistance, the 3× peak current): the manual gives only the 150 W rating, voltage and mass. The simulated mass (28.3 kg) is higher than the real 20 kg.
- **The patch-crossing test (Section 12-B) did not show a benefit of the correction** and logged no ground-truth pose; the benefit is shown only by the leg-4 overshoot in the patrols.
- Three planned runs per condition; contact physics is not repeatable; the completion rates (3 of 3 against 2 of 3) are not statistically significant.
- Heading at stations is not controlled (yaw tolerance 3.14 rad) yet the patrol checks it within 40°; one slip-ON run failed that check 0.14 m from the goal.
- Tour distances come from the localisation pose, not ground truth; the extra time on slip legs (S7: 67 s against 33 s) and the leg-12 aborts were not investigated.
- Simulation only; keyboard teleoperation, SLAM mapping and the 10 m obstacle arena were not re-run.

Future enhancements: drive the wheels by torque (joint force) so that slip follows from the motor and the floor; measure the real Scout Mini motor constants; add a slope world to use the 30° case of Table 5; repeat the patrol many more times over several patch layouts; investigate the leg-12 aborts; control the heading at stations; calibrate the odometry for skid-steer slip; monitor AMCL covariance instead of ground truth; and repeat the tests on the physical robot.

18. Conclusion

The Scout Mini simulation now has a drive-motor layer with the data of its four 150 W hub motors, a 24 V battery and a 1 : 1 gearbox, and a traction model of the floor. The calculation shows that the motors have a large reserve (cruise 20 % and a 4 m/s² start 66 % of the rated torque) while a low-friction patch ($\mu = 0.04$) allows only 11 N and 0.39 m/s², so any start or stop asked of Nav2 exceeds the grip by a factor of three. The calculated torque agrees with the simulation to within 0.01–0.05 N·m at cruise and in a spin, and six unit tests pass.

A slip controller that estimates the true speed from the IMU, limits the wheel command and corrects the odometry kept three of three planned patrols complete on the slip floor, against two of three without it, and prevented the overshoot of the corridor hub in every run. Direct goal sending kept the driven routes within about 2 % of the shortest grid paths. The results come from a small number of simulated runs; the steady-speed patch crossing of Section 12-B shows slip detection but no benefit of the correction.

19. References

[1] AgileX Robotics, *SCOUT MINI User Manual*, v1.0.1 (specification: four 150 W DC brushless motors, 24 V battery, 20 kg).

[2] S. Macenski, F. Martín, R. White, J. Ginés Clavero, “The Marathon 2: A Navigation System,” *IEEE/RSJ IROS*, 2020 (Nav2).

[3] T. Moore and D. Stouch, “A Generalized Extended Kalman Filter Implementation for the Robot Operating System,” *Proc. IAS-13*, 2014 (robot_localization).

[4] D. Fox, “KLD-Sampling: Adaptive Particle Filters,” *Advances in Neural Information Processing Systems 14*, 2001 (AMCL).

[5] R. Siegwart, I. R. Nourbakhsh, D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed., MIT Press, 2011 (wheel kinematics, slip).

[6] Open Robotics, *ROS 2 Jazzy Jalisco documentation*, and Gazebo Harmonic documentation (DiffDrive system, contact and friction), docs.ros.org and gazebosim.org.

[7] Nav2 documentation: Smac planner, regulated pure pursuit controller, velocity smoother, docs.nav2.org.